DATABASES

Indexing Deep Dive Visual Guide

Composite ordering, covering indexes, BRIN -- beyond 'just add an index'.

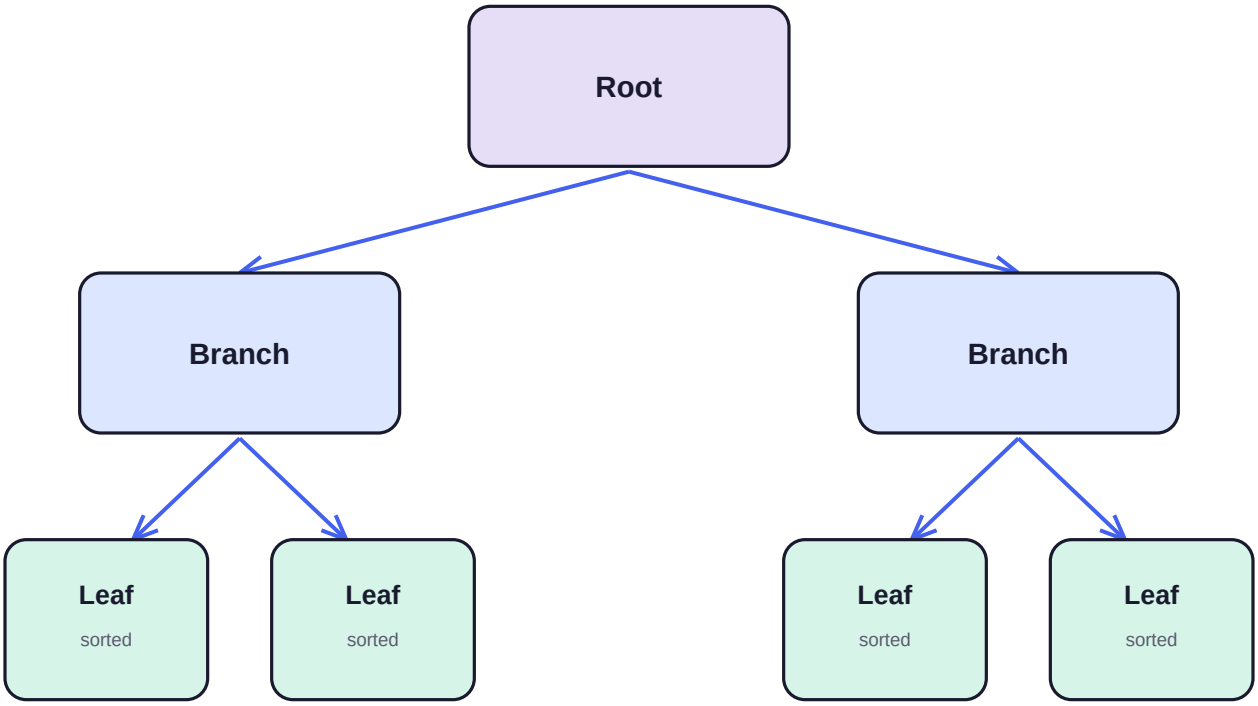
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

How B-Tree Indexes Work

Sorted structure, logarithmic lookups.



- Leaves hold sorted keys + row pointers; lookups walk root -> branch -> leaf: O(log n).
- Writes pay for index maintenance -- every index slows INSERT/UPDATE.
- Rule of thumb: index foreign keys, WHERE filters, and ORDER BY columns you actually use.

STEP 2/6

Composite Index Ordering

Column order decides which queries the index serves.

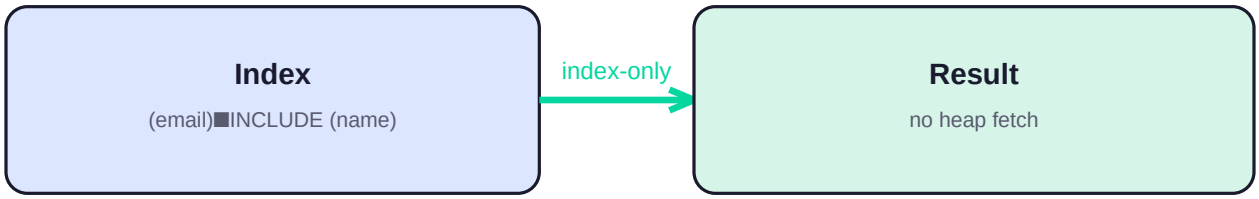
```
CREATE INDEX ON orders (customer_id, status, created_at);
-- serves:  WHERE customer_id = ?
-- serves:  WHERE customer_id = ? AND status = ?
-- serves:  WHERE customer_id = ? AND status = ? AND created_at > ?
-- MISSES:  WHERE status = ?   (leading column absent)
```

- Leftmost-prefix rule: the index works from the first column onward.
- Equality columns first, range/sort columns LAST -- a range stops the prefix.
- High-selectivity first is a myth for B-trees; equality-first ordering is what matters.

STEP 3/6

Covering Indexes

Answer the query from the index alone -- never touch the table.




```
CREATE INDEX ON users (email) INCLUDE (name);
SELECT name FROM users WHERE email = ?;  -- Index Only Scan
```

INCLUDE adds payload columns without sorting by them -- wider index, but zero table lookups.

STEP 4/6

Partial Indexes

Index only the rows you query.

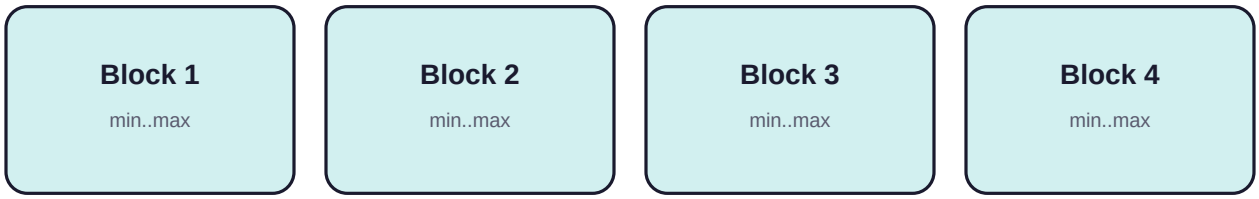
```
CREATE INDEX ON orders (created_at)
WHERE status = 'pending';
-- tiny index: only pending rows
-- queries with the same WHERE clause use it automatically
```

- Perfect for skewed predicates: active users, open tickets, unpaid invoices.
- Smaller index = faster writes, less memory, cheaper maintenance.
- The query's WHERE must imply the index predicate, or the planner ignores it.

STEP 5/6

BRIN for Time-Series

Block Range INdexes: tiny summaries for naturally ordered data.



BRIN stores one min/max per block -- KBs of index for TBs of data.

- Brilliant for append-only time series: each block's range is tight and ordered.
- Useless for random-ordered columns -- ranges overlap and nothing is pruned.
- B-tree on 1B rows: GBs. BRIN: KBs. The trade is precision for size.

STEP ✓I✓

Key Takeaways

Indexing Deep Dive -- Remember

- B-trees give O(log n) lookups; every index taxes writes -- index deliberately.
- Composite order: equality columns first, range/sort last; leftmost prefix rules.
- Covering indexes (INCLUDE) answer queries without touching the table.
- Partial indexes shrink the index to the rows you actually query.
- BRIN indexes time-series in kilobytes where B-trees need gigabytes.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev